

Aula 6 — Cross-Browser, CI/CD & Projeto Final

"Como provar pra liderança que QA gera valor?"

Prof. Jackson Smith Moisés Matias

PUC Minas IEC · 29/10/2026 · Última aula

Recap da disciplina

1. Fundamentos + Trophy/Honeycomb
2. Playwright avançado
3. SPA + PWA testing
4. Maestro mobile
5. **IA avançada**
6. **Hoje:** CI/CD + apresentação do projeto final

Agenda da aula (3h30)

- **90min** — Cross-browser matrix, Test Impact Analysis, quality gates, DORA metrics (teoria + demo guiada)
- **15min** — Intervalo
- **90min** — Apresentações do Projeto Final (ao vivo + Q&A)
- **15min** — Wrap-up e encerramento

Objetivos M6

- **Configurar** matriz cross-browser em Playwright
- **Aplicar** Test Impact Analysis pra reduzir tempo de feedback
- **Configurar** quality gates (SonarQube, Stryker mutation)
- **Apresentar** DORA metrics aplicadas a QA pra liderança
- **Concluir** projeto final defensavelmente

Cross-browser strategies

```
// playwright.config.ts
export default defineConfig({
  projects: [
    { name: 'chromium', use: { ...devices['Desktop Chrome'] } },
    { name: 'firefox', use: { ...devices['Desktop Firefox'] } },
    { name: 'webkit', use: { ...devices['Desktop Safari'] } },
    { name: 'mobile-chrome', use: { ...devices['Pixel 7'] } },
    { name: 'mobile-safari', use: { ...devices['iPhone 14'] } },
  ],
});
```

```
npx playwright test --project=chromium --project=webkit
```

Mobile farm — estratégia top 80%

Em vez de testar todos os devices, **cobrir top 10–15** que representam 80%+ usuários.

Fontes:

- StatCounter (público)
- App Annie / data.ai (premium)
- Próprio analytics do app (Firebase, Mixpanel, Amplitude)

| Variar OS versions é tão importante quanto modelos.

Test Impact Analysis (TIA)

Em vez de rodar suíte inteira em cada PR, **rodar só os testes afetados pelo diff.**

```
Diff modifica:  
src/checkout/utils.ts
```

```
TIA mapa de dependência:  
src/checkout/utils.ts → tests/checkout/*.spec.ts (47 testes)  
                      → tests/payment/*.spec.ts (12 testes)
```

```
Em vez de rodar 3000 testes, rodar 59.
```

| Ganho típico: 60–90% redução de tempo em PRs pequenos.

TIA — implementações

- **Microsoft TIA** (.NET ecosystem)
- **Bazel rules** (mono-repos)
- **Facebook Sapienz**
- **Jest --findRelatedTests** (limitado a unit)

Para Playwright/E2E, ainda menos maduro. Caminho: tags + diff de arquivos.

SonarQube — quality gates

```
# .github/workflows/sonar.yml
- name: SonarQube scan
  uses: sonarsource/sonarqube-scan-action@v2
  with:
    args: >
      -Dsonar.projectKey=myapp
      -Dsonar.organization=myorg

- name: SonarQube quality gate check
  uses: sonarsource/sonarqube-quality-gate-action@v1
  timeout-minutes: 5
  env:
    SONAR_TOKEN: ${ secrets.SONAR_TOKEN }
```

Falha PR se:

- Coverage < threshold
- Bugs encontrados
- Code smells acima do limite

Stryker — mutation testing

```
npm install --save-dev @stryker-mutator/core @stryker-mutator/jest-runner  
  
npx stryker init  
npx stryker run
```

```
// stryker.conf.json  
{  
  "testRunner": "jest",  
  "coverageAnalysis": "perTest",  
  "thresholds": { "high": 80, "low": 60, "break": 50 }  
}
```

Mutation score < 50% **falha o build**. Prova que testes verificam comportamento.

Por que mutation > line coverage

```
// Line coverage: 100%
function divide(a: number, b: number) {
  if (b === 0) throw new Error('Division by zero');
  return a / b;
}

test('divide works', () => {
  divide(10, 2); // executou linha – coverage 100%
});
```

Mutation score: 0%. Stryker introduz mutações:

- `b === 0` → `b !== 0` (teste passa, sem assertion)
- `a / b` → `a * b` (teste passa, sem assertion)

Coverage mente. Mutation prova.

DORA metrics — aplicação a QA

Quatro metrics chave, *Accelerate* (Forsgren, Humble, Kim 2018):

Metric	Mede	QA Influence
Deployment Frequency	Velocidade entrega	CI rápido + estável
Lead Time for Changes	Commit → produção	Suíte rápida + estável
Change Failure Rate	% deploys falham	KPI direto QA
MTTR	Tempo recuperação	Observabilidade boa

DORA — benchmarks 2024–2026

Categoria	DF	Lead Time	CFR	MTTR
Elite	On demand (multi/dia)	< 1h	< 15%	< 1h
High	Daily — weekly	1d–1w	15–20%	1–24h
Medium	Weekly — monthly	1w–1mo	20–30%	1–7d
Low	Monthly+	1mo+	30–60%+	7d+

| Apresentar DORA pra liderança = comunicar valor de QA.

Métricas que importam dia-a-dia

- **Flake rate** — alvo $< 5\%$
- **Escape rate** — bugs em produção / total de bugs
- **Lead time pra merge** — quanto demora um PR
- **Coverage** (linha + branch + mutation)
- **Test execution time** — minutes, não horas
- **Time-to-feedback** (CI duration)
- **Test reliability score** (consistência)

Pipeline CI/CD completo

```
name: CI
on: [pull_request]

jobs:
  unit:
    runs-on: ubuntu-latest
    steps:
      - run: npm test -- --coverage

  e2e:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        shard: [1/4, 2/4, 3/4, 4/4]
    steps:
      - run: npx playwright test --shard=${{ matrix.shard }}

  mobile:
    runs-on: macos-14
    steps:
      - run: maestro test flows/

  mutation:
    runs-on: ubuntu-latest
    steps:
      - run: npx stryker run

  lighthouse:
    runs-on: ubuntu-latest
    steps:
      - run: lhci autorun

  sonar:
    needs: [unit, e2e, mutation]
    runs-on: ubuntu-latest
    steps:
      - run: mvn sonar:sonar --batch-mode --settings sonar.xml --Dsonar.projectName=meq9-10-2026@v2
```

Slack notification — resultados

```
- name: Notify Slack
  if: always()
  uses: 8398a7/action-slack@v3
  with:
    status: ${{ job.status }}
    fields: repo,message,commit,author,action,eventName,ref
  env:
    SLACK_WEBHOOK_URL: ${{ secrets.SLACK_WEBHOOK_URL }}
```

Notificação estruturada com link pro relatório. Time vê falha sem precisar entrar no GitHub.

Lab — pipeline completo (90min)

Roteiro

1. Clone starter `modulo-06` (5min)
2. Configurar Playwright sharding 4-way + Maestro (15min)
3. SonarQube + Stryker quality gates (15min)
4. Lighthouse CI integrado (10min)
5. Visual AI baseline (Applitools ou pixelmatch) (10min)
6. Slack notification estruturada (10min)
7. Calcular DORA metrics simuladas pra apresentar (15min)
8. Discussão (10min)

Projeto Final — checklist

Suíte completa para app de referência (web SPA + PWA + mobile RN). 35pts.

Entregáveis:

1. **Suíte Playwright web** com 10+ E2E + visual regression — 8pts
2. **Suíte PWA** com offline + SW + Lighthouse CI — 5pts
3. **Suíte Maestro mobile** com 5+ flows iOS + Android — 7pts
4. **Componente IA não trivial** (1 das 3 opções) — 8pts
5. **CI/CD funcional** com sharding, retries, quality gates — 4pts
6. **Relatório técnico 10pgs** — 3pts

Projeto Final — opções IA

Escolha **uma**:

- (a) **Pipeline geração testes via Claude API** a partir de user story (com healing loop)
- (b) **Visual AI integrado** com baseline + análise de drift + classificação de mudança (intencional/regressão)
- (c) **Agent de exploratory testing** (Browser Use ou Computer Use) descobrindo bugs sem script

| Não vale: chamada simples a API LLM com prompt único e sem loop/análise.

Projeto Final — relatório (10pgs)

Estrutura recomendada:

1. **Estratégia adotada** (pirâmide vs trophy vs honeycomb — justificar)
2. **Cobertura por tipo** (unit/integration/E2E/visual)
3. **Métricas** (tempo de execução, flake rate, coverage)
4. **Lições aprendidas** (incluindo limites IA — ser crítico)
5. **Decisões arquiteturais** (ADRs curtos)
6. **Referências** — mínimo 8, sendo 3+ acadêmicas

ABNT ou IEEE.

Projeto Final — entrega

- Repo GitHub público com README de execução completo
- Workflow GitHub Actions verde (badge no README)
- Vídeo demo 5min explicando arquitetura + IA component
- Relatório em `docs/RELAT0RIO.md` (10 páginas)

Critério eliminatório: projeto final é obrigatório.

Wrap-up — encerramento

Vocês saem da disciplina com:

- Fundamentos: ISO 25010, pirâmide aplicada com critério
- Playwright + Maestro production-grade
- SPA, PWA testing avançado
- IA em testes — Visual AI, Claude API gen, agentes
- Pipeline CI/CD completo
- DORA metrics pra apresentar pra liderança

E uma bibliografia âncora pra visitar nos próximos anos.

Continuem aprendendo

- **Playwright** continua evoluindo rápido
- **Visual AI** vira commodity em times sérios
- **AI agents** (Computer Use, Browser Use) amadurecem mensalmente
- **DORA metrics** viram lingua franca de engenharia
- **Regulação privacy** (LGPD, EU AI Act) muda QA também

Feedback (sempre bem-vindo)

Quero saber:

- O que funcionou
- O que faltou
- O que posso melhorar

Formulário: link no Canvas.

| Sua resposta vira melhoria real na próxima oferta.

Obrigado!

Foi um prazer ensinar essa turma.

 jackson.96@gmail.com

 [linkedin.com/in/3jacksonsmith](https://www.linkedin.com/in/3jacksonsmith)

 github.com/jacksonsmith

Boa sorte com o projeto final e seus desafios de QA!